

De PHP à JavaScript : L'essentiel pour démarrer

Objectif : Maîtriser rapidement la syntaxe et les concepts de base de JavaScript en capitalisant sur vos connaissances en PHP.

1. Introduction à JavaScript

La bonne nouvelle ? Vous connaissez déjà la logique de programmation (les variables, les boucles, les conditions). Passer de PHP à JS n'est qu'une question de traduction : les concepts restent les mêmes, seule l'orthographe change !

2. Les Bases : Variables, Types, Affichage et Interpolation

A. Déclaration et Typage

En PHP, toute variable commence par un \$. En JavaScript, le \$ disparaît au profit de mots-clés définissant la nature de la variable : `let` (variable modifiable) ou `const` (constante). Comme PHP, JS est un langage à typage dynamique.

PHP :

```
<?php
$nom = "Dupont";
define("TVA", 20); // ou const TVA = 20;
$age = 19;
?>
```

JavaScript :

```
let nom = "Dupont";
const TVA = 20; // On ne peut plus la modifier ensuite !
let age = 19;
```

💡 Oubliez l'ancien mot-clé `var`. Aujourd'hui, on utilise `const` par défaut, et `let` uniquement si on sait que la valeur va changer au cours du script.

B. Les Types de données (Primitives)

JavaScript possède plusieurs types de données de base. Voici les plus courants :

- **String** : Chaînes de caractères (ex: "Bonjour").
- **Number** : Nombres (entiers ou décimaux, ex: 42 ou 3.14). *Note : JS ne distingue pas int et float.*
- **Boolean** : true ou false.
- **BigInt** : Pour les très grands nombres entiers.
- **Object** : Pour les structures complexes (tableaux, objets littéraux).

C. Le cas particulier : Null vs Undefined

Contrairement au PHP qui n'utilise que null, JavaScript utilise deux valeurs pour exprimer l'absence de donnée. C'est une nuance cruciale :

1. **undefined (Non défini)** : C'est une boîte qui a bien son étiquette (le nom de la variable), mais on ne sait pas encore ce qu'il y a dedans : elle est "brute" et non initialisée.
2. **null (Nul)** : C'est une boîte avec son étiquette, dans laquelle on a délibérément placé un panneau "C'est vide" pour indiquer qu'elle n'est pas censée contenir d'objet pour l'instant.

Comparaison concrète :

```
let test;
```

```
console.log(test); // Affiche : undefined (JS ne sait pas encore ce que c'est)
```

```
let resultat = null;
```

```
console.log(resultat); // Affiche : null (Le développeur dit : "C'est vide pour l'instant")
```

D. Affichage et Interpolation (Concaténation)

Fini les echo ou var_dump(). En JS, notre meilleur ami est la console du navigateur (F12).

Pour l'interpolation (insérer des variables dans du texte), JS utilise les "backticks" (`) à la place des guillemets doubles.

PHP :

```
<?php
echo "Bonjour $nom, tu as $age ans.";
var_dump($nom);
?>
```

JavaScript :

```
// Interpolation avec les backticks (Alt Gr + 7) et ${...}  
console.log(`Bonjour ${nom}, tu as ${age} ans.`);
```

```
// Pour inspecter une variable (équivalent de var_dump)  
console.log(nom);
```

3. Les Structures Conditionnelles

A. If, Else If, Else

Ici, vous êtes en terrain conquis ! La syntaxe est presque **strictement identique** à celle de PHP.

PHP :

```
<?php  
if ($age >= 18) {  
    echo "Majeur";  
} elseif ($age === 17) {  
    echo "Presque !";  
} else {  
    echo "Mineur";  
}  
?>
```

JavaScript :

```
if (age >= 18) {  
    console.log("Majeur");  
} else if (age === 17) { // Attention, c'est 'else if' en deux mots !  
    console.log("Presque !");  
} else {  
    console.log("Mineur");  
}
```

 **Le piège classique :** Comme en PHP, préférez TOUJOURS le triple égal === (qui vérifie la valeur ET le type) plutôt que le double égal == en JavaScript pour éviter des comportements imprévisibles (ex: 0 == "0" est vrai, mais 0 === "0" est faux).

B. L'opérateur Ternaire

L'opérateur ternaire permet d'écrire une condition simple sur une seule ligne. C'est l'équivalent direct du ternaire PHP.

PHP :

```
<?php
$statut = ($age >= 18) ? "Adulte" : "Mineur";
?>
```

JavaScript :

```
const statut = (age >= 18) ? "Adulte" : "Mineur";
console.log(statut);
```

4. Les Boucles (Sans les tableaux)

Même constat que pour les conditions : while, do...while et la boucle for classique s'écrivent exactement de la même manière. N'oubliez juste pas de déclarer votre compteur avec let !

La boucle FOR

PHP :

```
<?php
for ($i = 0; $i < 5; $i++) {
    echo $i;
}
?>
```

JavaScript :

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```

La boucle WHILE

PHP :

```
<?php
$j = 0;
while ($j < 3) {
    echo $j;
    $j++; // Ne pas oublier d'incrémenter pour éviter la boucle infinie !
}
?>
```

JavaScript :

```
let j = 0;
while (j < 3) {
    console.log(j);
    j++; // Exactement la même logique qu'en PHP
}
```

5. Les Tableaux Indexés

Un tableau indexé est une liste numérotée (de 0 à n). La manipulation change un peu car en JS, **tout est objet**. Un tableau possède donc des "méthodes" et des "propriétés" (comme .length ou .push()) plutôt que de passer par des fonctions externes comme en PHP (count(), array_push()).

PHP :

```
<?php
$langages = ["PHP", "SQL"];
$langages[] = "Python"; // Ajout

$taille = count($langages);

// Parcours
foreach ($langages as $lang) {
    echo $lang;
}
?>
```

JavaScript :

```
const langages = ["PHP", "SQL"];
langages.push("Python"); // Ajout avec la méthode push()

let taille = langages.length; // Propriété length (sans parenthèses)

// Parcours : On utilise la boucle "for...of"
for (const lang of langages) {
    console.log(lang);
}
```

6. Les Objets JavaScript vs Tableaux Associatifs PHP

C'est ici que se trouve **le plus grand changement conceptuel**. En PHP, pour lier des clés à des valeurs, on utilise un tableau associatif (=>). En JavaScript, on utilise des **Objets littéraux** (:).

A. Déclaration et Accès

PHP (Tableau associatif) :

```
<?php
$etudiant = [
    "nom" => "Martin",
    "prenom" => "Alice",
    "moyenne" => 14.5
];
echo $etudiant["prenom"]; // Alice
?>
```

JavaScript (Objet) :

```
const etudiant = {
    nom: "Martin",
    prenom: "Alice",
    moyenne: 14.5
};

// En JS, on accède aux propriétés par un point (.)
console.log(etudiant.prenom); // Alice
```

B. Le vocabulaire technique à maîtriser

En JavaScript, un objet est composé de plusieurs propriétés. Chaque propriété associe un nom appelé aussi clé à une valeur.

Terme JS	Définition	Équivalent PHP (Tableau Assoc.)
Objet littéral	Une structure délimitée par { }.	Le tableau associatif [].
Propriété	L'ensemble "Clé + Valeur" (ex: nom: "Martin").	Une ligne du tableau associatif.
Clé (ou Nom)	L'identifiant situé à gauche du : (ex: nom).	La clé du tableau.
Valeur	La donnée située à droite du : (ex: "Martin").	La valeur associée.

C. Parcourir un objet : la boucle for...in

Pour parcourir toutes les propriétés d'un objet, JS propose la boucle for...in. C'est le cousin germain du foreach (\$array as \$key => \$value) de PHP.

PHP :

```
<?php
foreach ($etudiant as $cle => $valeur) {
    echo "$cle : $valeur";
}
?>
```

JavaScript :

```
for (const cle in etudiant) {
    // On accède à la valeur via la syntaxe entre crochets []
    console.log(` ${cle} : ${etudiant[cle]} `);
}
```

💡 Notez qu'en JS, pour récupérer la valeur dynamiquement avec une variable (cle), on doit utiliser les crochets `etudiant[cle]`. Faire `etudiant.cle` chercherait littéralement une propriété nommée "cle" dans l'objet.

7. Les Tableaux d'Objets vs Tableaux à 2 dimensions PHP

Dans la vraie vie (ex: lors d'une requête vers une API ou une BDD), on manipule souvent des listes d'éléments complexes.

En PHP : un tableau contenant des tableaux associatifs.

En JS : un tableau contenant des objets.

PHP :

```
<?php
$utilisateurs = [
    ["id" => 1, "nom" => "Dupont"],
    ["id" => 2, "nom" => "Durand"]
];

foreach ($utilisateurs as $user) {
    echo $user["nom"];
}
?>
```

JavaScript :

```
const utilisateurs = [
    { id: 1, nom: "Dupont" },
    { id: 2, nom: "Durand" }
];

for (const user of utilisateurs) {
    console.log(user.nom); // Beaucoup plus lisible qu'en PHP, non ?
}
```


8. Les Fonctions "Classiques"

Pas de grand bouleversement ici. Le mot-clé est le même, la logique des paramètres par défaut est la même.

PHP :

```
<?php
function saluer($nom = "Inconnu") {
    return "Salut $nom !";
}
echo saluer("Alice");
?>
```

JavaScript :

```
function saluer(nom = "Inconnu") {
    return `Salut ${nom} !`;
}
console.log(saluer("Alice"));
```

9. Manipulation des Chaînes de Caractères

Tout comme les tableaux, les chaînes de caractères (Strings) en JavaScript se manipulent comme des objets, avec leurs propres méthodes, là où PHP utilise des fonctions globales.

Action	En PHP	En JavaScript
Longueur	strlen(\$chaine)	chaine.length
Majuscules	strtoupper(\$chaine)	chaine.toUpperCase()
Minuscules	strtolower(\$chaine)	chaine.toLowerCase()
Remplacer	str_replace("a", "b", \$ch)	chaine.replace("a", "b")
Découper	explode(",", \$chaine)	chaine.split(",")

Exemple JavaScript :

```
let phrase = "Bonjour les SIO";
console.log(phrase.toUpperCase()); // "BONJOUR LES SIO"
console.log(phrase.split(" ")); // ["Bonjour", "les", "SIO"]
```

10. Fonctions Anonymes et Fléchées

JavaScript possède des concepts d'écriture de fonctions très puissants et extrêmement utilisés en entreprise qu'il faut maîtriser.

Les Fonctions Anonymes

En JS, une fonction est une "valeur" comme une autre. On peut donc créer une fonction sans nom et la ranger directement dans une variable.

```
const faireUnCalcul = function(a, b) {  
  return a + b;  
};  
  
console.log(faireUnCalcul(5, 10)); // Affiche 15
```

Les Fonctions Fléchées (Arrow Functions)

Apparues plus récemment, elles permettent d'écrire des fonctions de manière beaucoup plus courte et élégante. Le mot-clé `function` disparaît au profit d'une "flèche" `=>`.

```
// Version classique anonyme  
const multiplier = function(a, b) {  
  return a * b;  
};  
  
// Version fléchée 🚀  
const multiplierFleche = (a, b) => {  
  return a * b;  
};  
  
// Si la fonction ne fait QUE retourner une valeur (sur une seule ligne),  
// on peut même enlever les accolades et le mot-clé 'return' :  
const multiplierUltraCourt = (a, b) => a * b;  
  
console.log(multiplierUltraCourt(4, 5)); // Affiche 20
```

🔗 **Pourquoi c'est important ?** Les fonctions fléchées sont partout en JavaScript moderne (React, Node.js, etc.). Entraînez-vous à lire et à écrire cette syntaxe !

11. Programmation Fonctionnelle avec les Tableaux (Les méthodes avancées)

En PHP, on utilise massivement les boucles foreach pour manipuler les tableaux. En JavaScript moderne, on préfère utiliser l'**approche fonctionnelle**. Couplées aux fonctions fléchées vues précédemment, ces méthodes (forEach, map, filter) rendent le code beaucoup plus concis.

Prenons un tableau d'objets pour nos exemples :

```
const etudiants = [  
  { nom: "Alice", note: 15 },  
  { nom: "Bob", note: 8 },  
  { nom: "Charlie", note: 12 }  
];
```

A. Parcourir : forEach()

Sert à exécuter une action pour chaque élément (ne retourne rien).

En procédural (Classique) :

```
for (let etudiant of etudiants) {  
  console.log(etudiant.nom);  
}
```

En fonctionnel :

```
etudiants.forEach(etudiant => console.log(etudiant.nom));
```

B. Filtrer : filter()

Sert à créer un **nouveau tableau** en ne gardant que les éléments qui valident une condition.

En procédural :

```
const admis = [];  
for (let etudiant of etudiants) {  
  if (etudiant.note >= 10) {  
    admis.push(etudiant);  
  }  
}
```

En fonctionnel :

```
const admis = etudiants.filter(etudiant => etudiant.note >= 10);  
// Résultat : [{nom: "Alice", note: 15}, {nom: "Charlie", note: 12}]
```

C. Transformer : map()

Sert à créer un **nouveau tableau** en transformant chaque élément du tableau de départ.

En procédural :

```
const nomsSeuls = [];  
for (let etudiant of etudiants) {  
  nomsSeuls.push(etudiant.nom);  
}
```

En fonctionnel :

```
const nomsSeuls = etudiants.map(etudiant => etudiant.nom);  
// Résultat : ["Alice", "Bob", "Charlie"]
```

D. La puissance du Chaînage (Combiner les méthodes)

La véritable magie de JavaScript réside dans le fait de pouvoir **chaîner** ces méthodes, car filter et map retournent de nouveaux tableaux.

Objectif : Récupérer uniquement les noms des étudiants ayant eu la moyenne.

```
const nomsDesAdmis = etudiants  
  .filter(etudiant => etudiant.note >= 10) // 1. On filtre les admis  
  .map(etudiant => etudiant.nom);          // 2. On extrait juste leurs noms  
  
console.log(nomsDesAdmis); // Affiche ["Alice", "Charlie"]
```

💡 Ces trois méthodes (forEach, map, filter) sont vitales si vous poursuivez sur des frameworks comme React ou Vue.js. Prenez le temps de bien comprendre la différence : forEach pour agir, filter pour filtrer, map pour transformer !